CHAPTER 4

Total Store Order and the x86 Memory Model

A widely implemented memory consistency model is *total store order (TSO)*. TSO was first introduced by SPARC and, more importantly, appears to match the memory consistency model of the widely used x86 architecture. RISC-V also supports a TSO extension, RVTSO, in part to aid porting of code originally written for x86 or SPARC architectures. This chapter presents this important consistency model using a pattern similar to that in the previous chapter on sequential consistency. We first motivate TSO/x86 (Section 4.1) in part by pointing out limitations of SC. We then present TSO/x86 at an intuitive level (Section 4.2) before describing it more formally (Section 4.3), explaining how systems implement TSO/x86, including atomic instructions and instructions used to enforce ordering between instructions (Section 4.4). We conclude by discussing other resources for learning more about TSO/x86 (Section 4.5) and comparing TSO/x86 and SC (Section 4.6).

4.1 MOTIVATION FOR TSO/X86

Processor cores have long used *write (store) buffers* to hold committed (retired) stores until the rest of the memory system could process the stores. A store enters the write buffer when the store commits, and a store exits the write buffer when the block to be written is in the cache in a read-write coherence state. Significantly, a store can enter the write buffer before the cache has obtained read-write coherence permissions for the block to be written; the write buffer thus hides the latency of servicing a store miss. Because stores are common, being able to avoid stalling on most of them is an important benefit. Moreover, it seems sensible to not stall the core because the core does not need anything, as the store seeks to update memory but not core state.

For a single-core processor, a write buffer can be made architecturally invisible by ensuring that a load to address A returns the value of the most recent store to A even if one or more stores to A are in the write buffer. This is typically done by either *bypassing* the value of the most recent store to A to the load from A, where "most recent" is determined by program order, or by stalling a load of A if a store to A is in the write buffer.

When building a multicore processor, it seems natural to use multiple cores, each with its own bypassing write buffer, and assume that the write buffers continue to be architecturally invisible.

Table 4.1: Can both r1 and r2 be set to 0?

Core C1	Core C2	Comments
S1: x = NEW; L2: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */

This assumption is wrong. Consider the example code in Table 4.1 (which is the same as Table 3.3 in the previous chapter). Assume a multicore processor with in-order cores, where each core has a single-entry write buffer and executes the code in the following sequence.

- Core C1 executes store S1, but buffers the newly stored NEW value in its write buffer.
- Likewise, core C2 executes store S2 and holds the newly stored NEW value in its write buffer.
- Next, both cores perform their respective loads, L1 and L2, and obtain the old values of 0.
- Finally, both cores' write buffers update memory with the newly stored values NEW.

The net result is that (r1, r2) = (0, 0). As we saw in the previous chapter, this is an execution result forbidden by SC. Without write buffers, the hardware is SC, but with write buffers, it is not, making write buffers architecturally visible in a multicore processor.

One response to write buffers being visible would be to turn them off, but vendors have been loath to do this because of the potential performance impact. Another option is to use aggressive, speculative SC implementations that make write buffers invisible again, but doing so adds complexity and can waste power to both detect violations and handle mis-speculations.

The option chosen by SPARC and later x86 was to abandon SC in favor of a memory consistency model that allows straightforward use of a first-in–first-out (FIFO) write buffer at each core. The new model, TSO, allows the outcome "(r1, r2) = (0, 0)." This model astonishes some people but, it turns out, behaves like SC for most programming idioms and is well defined in all cases.

4.2 BASIC IDEA OF TSO/X86

As execution proceeds, SC requires that each core preserves the program order of its loads and stores for all four combinations of consecutive operations:

- Load → Load
- Load → Store
- Store → Store

- Store → Load /* Included for SC but omitted for TSO */

TSO includes the first three constraints but not the fourth. This omission does not matter for most programs. Table 4.2 repeats the example program of Table 3.1 in the previous chapter. In this case, TSO allows the same executions as SC because TSO preserves the order of core C1's two stores and core C2's two (or more) loads. Figure 4.1 (the same as Figure 3.2 in the previous chapter) illustrates the execution of this program.

Table 4.2: Should r2 always be set to NEW?

Core C1	Core C2	Comments
S1: Store data = NEW; S2: Store flag = SET;	L1: Load r1 = flag B1: if (r1 ≠ SET) goto L1; L2: Load r2=data;	/* Initially, data = 0 & flag ≠ SET */ /* L1 & B1 may repeat many times */

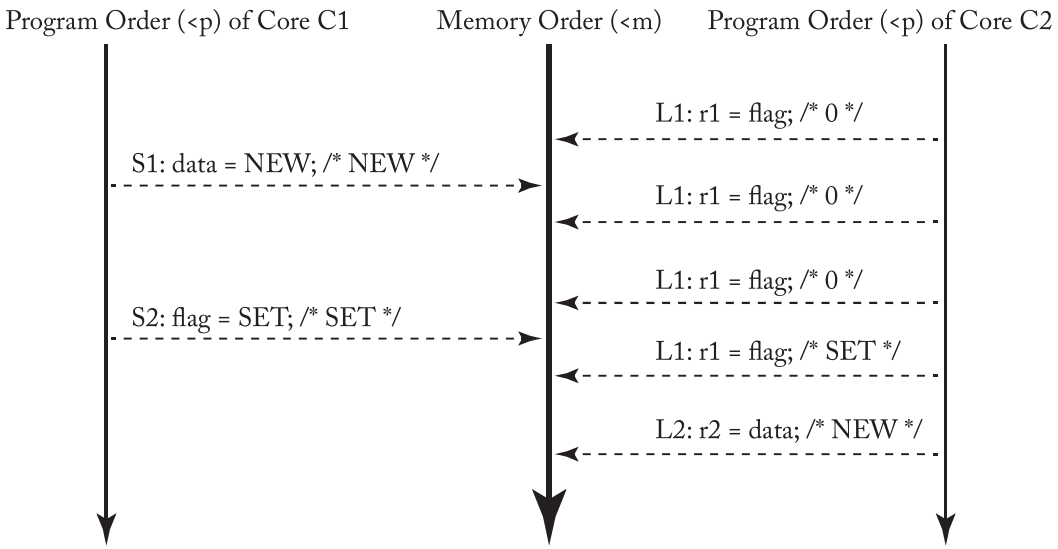


Figure 4.1: A TSO execution of Table 4.2's program.

More generally, TSO behaves the same as SC for common programming idioms that follow:

- C1 loads and stores to memory locations D1, . . ., D*n* (often data),
- C1 stores to F (often a synchronization flag) to indicate that the above work is complete,

- C2 loads from F to observe the above work is complete (sometimes spinning first and often using a RMW instruction), and
- C2 loads and stores to some or all of the memory locations D1, . . ., D*n*.

TSO, however, allows some non-SC executions. Under TSO, the program from Table 4.1 (repeat of Table 3.3 from the last chapter) allows all four outcomes depicted in Figure 4.2. Under SC, only the first three are legal outcomes (as depicted in Figure 3.3 of the last chapter). The execution in Figure 4.2d illustrates an execution that conforms to TSO but violates SC by not honoring the fourth (i.e., Store → Load) constraint. Omitting the fourth constraint allows each core to use a write buffer. Note that the third constraint means that the write buffer must be FIFO (and not, for example, coalescing) to preserve store–store order.

Programmers (or compilers) can prevent the execution in Figure 4.2d by inserting a FENCE instruction between S1 and L1 on core C1 and between S2 and L2 on core C2. Executing a FENCE on core Ci ensures that Ci's memory operations before the FENCE (in program order) get placed in memory order before Ci's memory operations after the FENCE. FENCEs (a.k.a. memory barriers) are rarely used by programmers using TSO because TSO "does the right thing" for most programs. Nevertheless, FENCEs play an important role for the relaxed models discussed in the next chapter.

TSO does allow some non-intuitive execution results. Table 4.3 illustrates a modified version of the program in Table 4.1 in which cores C1 and C2 make local copies of x and y, respectively. Many programmers might assume that if both r2 and r4 equal 0, then r1 and r3 should also be 0 because the stores S1 and S2 must be inserted into memory order after the loads L2 and L4. However, Figure 4.3 illustrates an execution that shows r1 and r3 bypassing the value NEW from the per-core write buffers. In fact, to preserve single-thread sequential semantics, each core must see the effect of its own store in program order, even though the store is not yet observed by other cores. Thus, under all TSO executions, the local copies r1 and r3 will always be set to the NEW value.

Table 4.3: Can r1 or r3 be set to 0?

Core C1	Core C2	Comments
S1: x = NEW;	S2: y = NEW;	/* Initially, x = 0 & y = 0 */
L1: r1 = x;	L3: r3 = y;	
L2: r2 = y;	L4: r4 = x;	/* Assume r2 = 0 & r4 =0 */

4.3 A LITTLE TSO/X86 FORMALISM

In this section we define TSO more precisely with a definition that makes only three changes to the SC definition of Section 3.5.

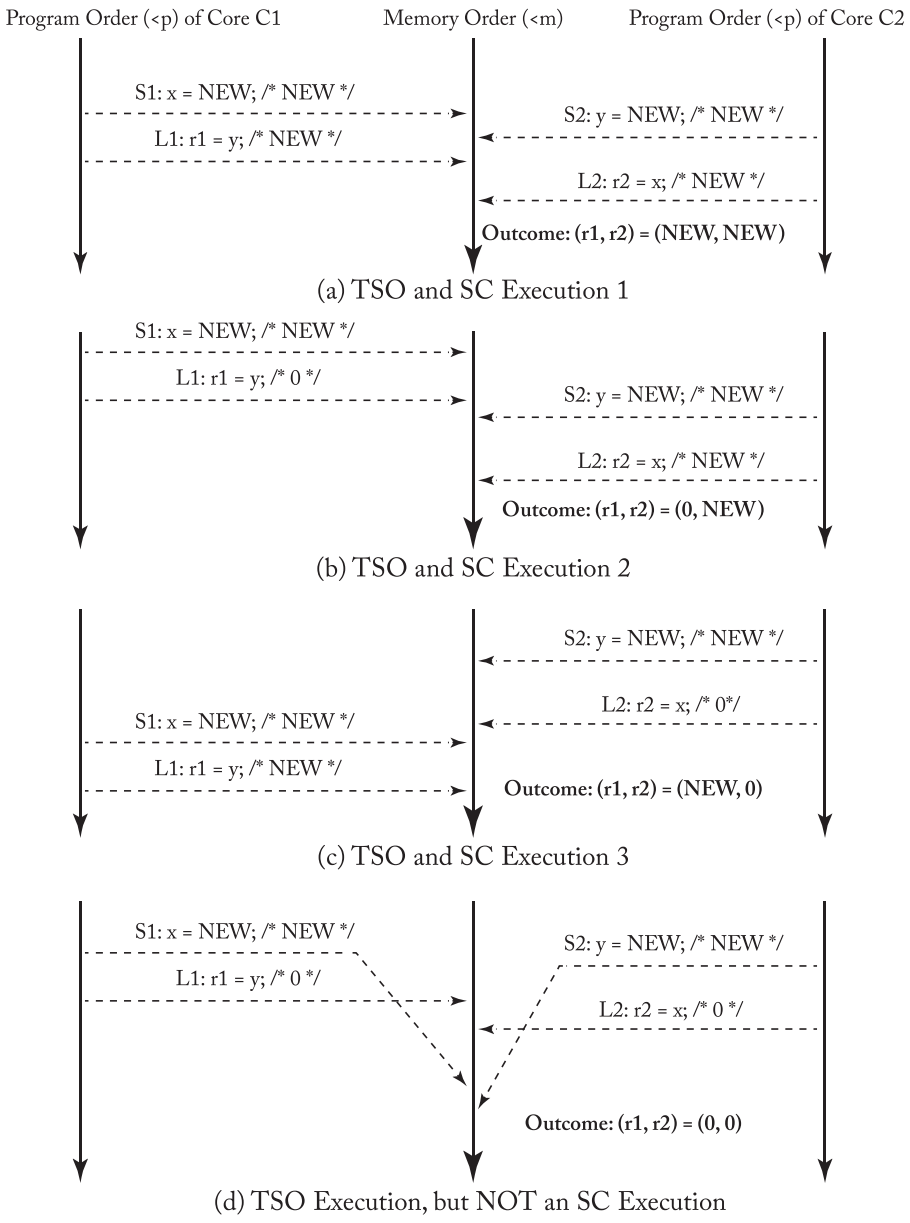


Figure 4.2: Four alternative TSO executions of Table 4.1's program.

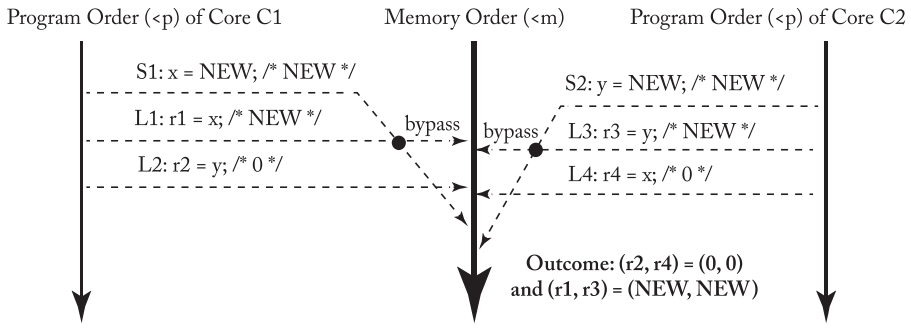


Figure 4.3: A TSO execution of Table 4.3's program (with "bypassing").

A **TSO execution** requires the following.

1. All cores insert their loads and stores into the memory order <m respecting their program order, regardless of whether they are to the same or different addresses (i.e., a==b or a!=b). There are four cases:
   - If L(a) <p L(b) ⇒ L(a) <m L(b) /* Load → Load */
   - If L(a) <p S(b) ⇒ L(a) <m S(b) /* Load → Store */
   - If S(a) <p S(b) ⇒ S(a) <m S(b) /* Store → Store */
   - ~~**If S(a) <p L(b) ⇒ S(a) <m L(b) /* Store → Load */**~~ **/* Change 1: Enable FIFO Write Buffer */**
2. Every load gets its value from the last store before it to the same address:
   - ~~**Value of L(a) = Value of MAX $_{<m}$ {S(a) | S(a) <m L(a)}**~~ **/* Change 2: Need Bypassing */**
   - Value of L(a) = Value of MAX $_{<m}$ {S(a) | S(a) <m L(a) **or S(a) <p L(a)**}

This last mind-bending equation says that the value of a load is the value of the last store to the same address that is either (a) before it in memory order or (b) before it in program order (but possibly after it in memory order), with option (b) taking precedence (i.e., write buffer bypassing overrides the rest of the memory system).

3. Part (1) must be augmented to define FENCEs: /* Change 3: FENCEs Order Everything */
   - If L(a) <p FENCE ⇒ L(a) <m FENCE /* Load → FENCE */
   - If S(a) <p FENCE ⇒ S(a) <m FENCE /* Store → FENCE */
   - If FENCE <p FENCE ⇒ FENCE <m FENCE /* FENCE → FENCE */

- If FENCE <p L(a) ⇒ FENCE <m L(a) /* FENCE → Load */
- If FENCE <p S(a) ⇒ FENCE <m S(a) /* FENCE → Store */

Because TSO already requires all but the Store → Load order, one can alternatively define TSO FENCEs as only ordering:

- If S(a) <p FENCE ⇒ S(a) <m FENCE /* Store → FENCE */
- If FENCE <p L(a) ⇒ FENCE <m L(a) /* FENCE → Load */

We choose to have TSO FENCEs redundantly order everything because doing so does not hurt and makes them like the FENCEs we define for more relaxed models in the next chapter.

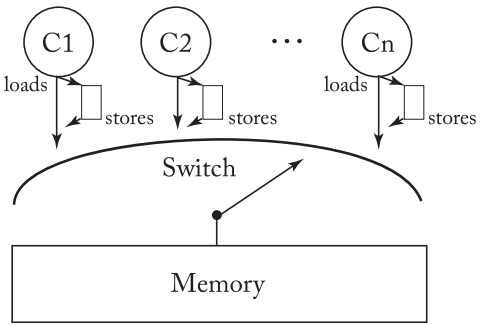
We summarize TSO's ordering rules in Table 4.4. This table has two important differences from the analogous table for SC (Table 3.4). First, if Operation #1 is a store and Operation #2 is a load, the entry at that intersection is a "B" instead of an "X"; if these operations are to the same address, the load must obtain the value just stored even if the operations enter memory order out of program order. Second, the table includes FENCEs, which were not necessary in SC; an SC system behaves as if there is already a FENCE before and after every operation.

Table 4.4: TSO ordering rules. An "X" denotes an enforced ordering. A "B" denotes that bypassing is required if the operations are to the same address. Entries that are different from the SC ordering rules are shaded and shown in bold.

		Operation 2			
		Load	Store	RMW	FENCE
Operation 1	Load	X	X	X	**X**
	Store	**B**	X	X	**X**
	RMW	X	X	X	**X**
	FENCE	**X**	**X**	**X**	**X**

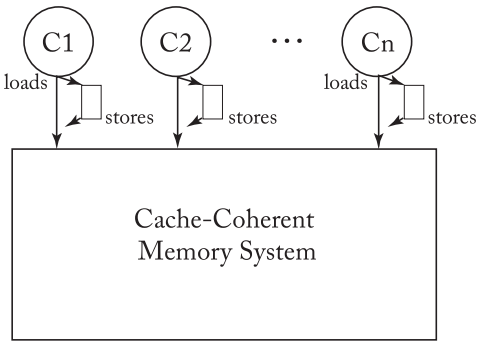
It is widely believed that the x86 memory model is equivalent to TSO (for normal cacheable memory and normal instructions), but to the best of our knowledge, neither AMD nor Intel have guaranteed this or released a formal x86 memory model specification. AMD and Intel publicly define the x86 memory model with examples and prose in a process that is well summarized in Section 2 of Sewell et al. [15]. All examples conform to TSO, and all prose seems consistent with TSO. This equivalence can be proven only if a public, formal description of the x86 memory model were made available. This equivalence could be disproved if counterexample(s) showed an x86 execution not allowed by TSO, a TSO execution not allowed by x86, or both.

That x86 is equivalent to TSO is supported by Sewell et al. [15], summarized in CACM with more details elsewhere [9, 14]. In particular, the authors propose the x86-TSO model. The model has two forms that the authors prove equivalent. The first form provides an abstract machine that resembles Figure 4.4a of the next section with the addition of a single global lock for modeling x86 LOCK'd instructions. The second form is a labeled transition system. The first form makes the model accessible to practitioners while the latter eases formal proofs. On one hand, x86-TSO appears consistent with the informal rules and litmus tests in x86 specifications. On the other hand, empirical tests on several AMD and Intel platforms did not reveal any violations of the x86-TSO model (but this is no proof that they cannot). In summary, like Sewell et al., we urge creators of x86 hardware and software to adopt the unambiguous and accessible x86-TSO model.



This implementation is the same as for Figure 3.4, except that each core Ci has a FIFO write buffer that buffers stores until they go to memory.

(a) A TSO implementation using a switch



This TSO implementation replaces the switch above with a cache-coherent memory system in a manner analogous to what was done for SC.

(b) A TSO implementation using cache coherence

Figure 4.4: Two TSO implementations.

4.4 IMPLEMENTING TSO/X86

The implementation story for TSO/x86 is similar to SC with the addition of per-core FIFO write buffers. Figure 4.4a updates the switch of Figure 3.4 to accommodate TSO and operates as follows.

- Loads and stores leave each core in that core's program order <p.
- A load either bypasses a value from the write buffer or awaits the switch as before.
- A store enters the tail of the FIFO write buffer or stalls the core if the buffer is full.
- When the switch selects core Ci, it performs either the next load or the store at the head of the write buffer.

In Section 3.7, we showed that, for SC, the switch can be replaced by a cache coherent memory system and then argued that cores could be speculative and/or multithreaded and that nonbinding prefetches could be initiated by cores, caches, or software.

As illustrated in Figure 4.4b, the *same* argument holds for TSO with a FIFO writer buffer interposed between each core and the cache-coherent memory system. Thus, aside from the write buffer, all the previous SC implementation discussion holds for TSO and provides a way to build TSO implementations. Moreover, most current TSO implementations seem to use only the above approach: take an SC implementation and insert write buffers.

Regarding the write buffer, the literature and product space for how exactly speculative cores implement them is beyond the scope of this chapter. For example, microarchitectures can physically combine the store queue (uncommitted stores) and write buffer (committed stores), and/or physically separate load and store queues.

Finally, multithreading introduces a subtle write buffer issue for TSO. TSO write buffers are logically private to each thread context (virtual core). Thus, on a multithreaded core, one thread context should never bypass from the write buffer of another thread context. This logical separation can be implemented with per-thread-context write buffers or, more commonly, by using a shared write buffer with entries tagged by thread-context identifiers that permit bypassing only when tags match.

Flashback to Quiz Question 4: In a TSO system with multithreaded cores, threads may bypass values out of the write buffer, regardless of which thread wrote the value. *True or false?* **Answer:** *False!* A thread may bypass values that it has written, but other threads may not see the value until the store is inserted into the memory order.

4.4.1 IMPLEMENTING ATOMIC INSTRUCTIONS

The implementation issues for atomic RMW instructions in TSO are similar to those for atomic instructions for SC. The key difference is that TSO allows loads to pass (i.e., be ordered before)

earlier stores that have been written to a write buffer. The impact on RMWs is that the "write" (i.e., store) may be written to the write buffer.

To understand the implementation of atomic RMWs in TSO, we consider the RMW as a load immediately followed by a store.

The load part of the RMW cannot pass earlier loads due to TSO's ordering rules. It might at first appear that the load part of the RMW could pass earlier stores in the write buffer, but this is not legal. If the load part of the RMW passes an earlier store, then the store part of the RMW would also have to pass the earlier store because the RMW is an atomic pair. But because stores are not allowed to pass each other in TSO, the load part of the RMW cannot pass an earlier store either.

These ordering constraints on RMWs impact the implementation. Because the load part of the RMW cannot be performed until earlier stores have been ordered (i.e., exited the write buffer), the atomic RMW effectively drains the write buffer before it can perform the load part of the RMW. Furthermore, to ensure that the store part can be ordered immediately after the load part, the load part requires read-write coherence permissions, not just the read permissions that suffice for normal loads. Lastly, to guarantee atomicity for the RMW, the cache controller may not relinquish coherence permission to the block between the load and the store.

More optimized implementations of RMWs are possible. For example, the write buffer does not need to be drained as long as (a) every entry already in the write buffer has read-write permission in the cache and maintains the read-write permission in the cache until the RMW commits, and (b) the core performs MIPS R10000-style checking of load speculation (Section 3.8). Logically, all of the earlier stores and loads would then commit as a unit (sometimes called a "chunk") immediately before the RMW.

4.4.2 IMPLEMENTING FENCES

Systems that support TSO do not provide ordering between a store and a subsequent (in program order) load, although they do require the load to get the value of the earlier store. In situations in which the programmer wants those instructions to be ordered, the programmer must explicitly specify that ordering by putting a FENCE instruction between the store and the subsequent load. The semantics of the FENCE specify that all instructions before the FENCE in program order must be ordered before any instructions after the FENCE in program order. For systems that support TSO, the FENCE thus prohibits a load from bypassing an earlier store. In Table 4.5, we revisit the example from Table 4.1, but we have added two FENCE instructions that were not present earlier. Without these FENCEs, the two loads (L1 and L2) can bypass the two stores (S1 and S2), leading to an execution in which r1 and r2 both get set to zero. The added FENCEs prohibit that reordering and thus prohibit that execution.

Because TSO permits only one type of reordering, FENCEs are fairly infrequent and the implementation of FENCE instructions is not too critical. A simple implementation—such as

Table 4.5: Can both r1 and r2 be set to 0?

Core C1	Core C2	Comments
S1: x = NEW;	S2: y = NEW;	/* Initially, x = 0 & y = 0 */
FENCE	**FENCE**	
L1: r1 = y;	L2: r2 = x;	

draining the write buffer when a FENCE is executed and not permitting subsequent loads to execute until an earlier FENCE has committed—may provide acceptable performance.

However, for consistency models that permit far more reordering (discussed in the next chapter), FENCE instructions are more frequent and their implementations can have a significant impact on performance.

Sidebar: Non-speculative TSO Optimizations

This sidebar describes some advanced non-speculative TSO optimizations.

Non-speculative TSO reordering. There have been papers that have shown that both loads [11, 12] and stores [13] can be reordered non-speculatively, while still enforcing TSO, using coherence delaying. As mentioned earlier, the key challenge is to ensure that the delays do not cause cyclic dependencies that lead to a deadlock, which all of the above papers address.

RMW without write buffer drain. In Section 4.4.1, we showed how to move the write buffer drain off the critical path using speculation. Rajaram et al. [10] show that the same effect can be achieved non-speculatively if the atomicity semantics of an RMW are redefined. Recall that for the RMW to be atomic, we earlier mandated that the read and the write operations of the RMW must appear consecutively in the TSO global memory order. Consider the following relaxation in which an RMW is deemed to be atomic as long as writes to the same address as that of the RMW do not appear between the read and write in the global memory order. Note that this relaxed definition matches the intuitive definition of an RMW and is sufficient for them to be used in synchronization situations. At the same time, it allows for an RMW implementation in which the load part of the RMW can bypass the earlier stores in the write buffer without requiring MIPS R10000-style load speculation checks. However, ensuring RMW atomicity necessitates coherence delaying until the write buffer drains, which introduces a deadlock hazard. Rajaram et al. [10] show how deadlocks can be avoided.

Reordering past a FENCE. There have been several papers that have proposed optimized FENCE implementations [3, 4, 6, 8] that enable memory operations following

the FENCE to be non-speculatively retired before those that precede the FENCE. These techniques either use coherence delays or predecessor serialization or a combination of both.

4.5 FURTHER READING REGARDING TSO

Collier [2] characterized alternative memory consistency models, including that of the IBM System/370, via a model in which each core has a full copy of memory, its loads read from the local copy, and its writes update all copies according to some restrictions that define a model. Were TSO defined with this model, each store would write its own core's memory copy immediately and then possibly later update all other memories together.

Goodman [7] publicly discussed the idea of *processor consistency (PC)*, wherein a core's stores reach other cores in order but do not necessarily reach other cores at the same "time." Gharachorloo et al. [5] more precisely define PC. TSO and x86-TSO are special cases of PC in which each core sees its own store immediately, and when any other cores see a store, all other cores see it. This property is called *write atomicity* in the next chapter (Section 5.5).

To the best of our knowledge, TSO was first formally defined by Sindhu et al. [16]. As discussed, in Section 4.3, Sewell et al. [9, 14, 15] propose and formalize the x86-TSO model that appears consistent with AMD and Intel x86 documentation and current implementations.

4.6 COMPARING SC AND TSO

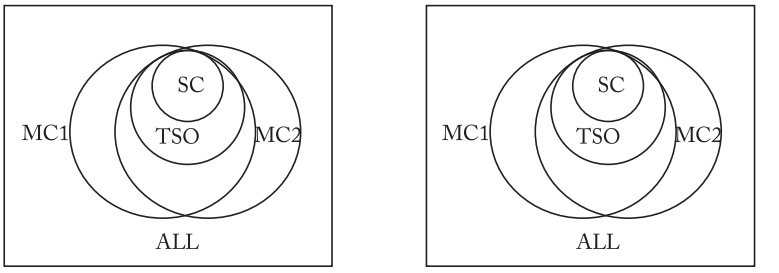
Now that we have seen two memory consistency models, we can compare them. How do SC, TSO, etc., relate?

- *Executions*: SC executions are a proper subset of TSO executions; all SC executions are TSO executions, while some TSO executions are SC executions and some are not. See the Venn diagram in Figure 4.5a.

- *Implementations*: Implementations follow the same rules: SC implementations are a proper subset of TSO implementations. See Figure 4.5b, which is the same as Figure 4.5a.

More generally, a memory consistency model Y is strictly more *relaxed* (*weaker*) than a memory consistency model X if all X executions are also Y executions, but not vice versa. If Y is more relaxed than X, then it follows that all X implementations are also Y implementations. It is also possible that two memory consistency models are incomparable because both allow executions precluded by the other.

As Figure 4.5 depicts, TSO is more relaxed than SC but less relaxed than incomparable models MC1 and MC2. In the next chapter, we will see candidates for MC1 and MC2, including a case study for the IBM Power memory consistency model.



(a) Executions (b) Implementations (same as (a))

Figure 4.5: Comparing memory consistency models.

What is a Good Memory Consistency Model?

A good memory consistency model should possess Sarita Adve's 3Ps [1] plus our fourth P:

- *Programmability*: A good model should make it (relatively) easy to write multithreaded programs. The model should be intuitive to most users, even those who have not read the details. It should be precise, so that experts can push the envelope of what is allowed.
- *Performance*: A good model should facilitate high-performance implementations at reasonable power, cost, etc. It should give implementors broad latitude in options.
- *Portability*: A good model would be adopted widely or at least provide backward compatibility or the ability to translate among models.
- *Precision*: A good model should be precisely defined, usually with mathematics. Natural languages are too ambiguous to enable experts to push the envelope of what is allowed.

How good are SC and TSO?

Using these 4Ps:

- *Programmability*: SC is the most intuitive. TSO is close because it acts like SC for common programming idioms. Nevertheless, subtle non-SC executions can bite programmers and tool authors.
- *Performance*: For simple cores, TSO can offer better performance than SC, but the difference can be made small with speculation.
- *Portability*: SC is widely understood, while TSO is widely adopted.
- *Precise*: SC and TSO are formally defined.

The bottom line is that SC and TSO are pretty close, especially compared with the more complex and more relaxed memory consistency models discussed in the next chapter.